

CSCE 608 DATABASE SYSTEMS · MODULE 07 · LECTURE NOTES

Query Execution

Turning a plan into rows, efficiently.

MODULE OUTCOMES

- Explain the iterator model and its cost.
- Compare iterator, materialisation, and vectorised execution.
- Explain query compilation and when it pays.
- Explain pipeline breakers and why they matter.
- Reason about parallel query execution.

1 The iterator model

The Volcano or iterator model, introduced by Graefe, gives every operator the same three-method interface. A query plan is a tree of such operators, and execution consists of repeatedly calling `next()` on the root.

```
struct Operator {
    virtual void open() = 0;
    virtual Tuple next() = 0;           // one tuple, or EOF
    virtual void close() = 0;
};

struct Filter : Operator {
    Tuple next() override {
        while (Tuple t = child->next())    // PULL from below
            if (pred->eval(t)) return t;
        return EOF;
    }
};
```

WHY THE MODEL LASTED, AND WHERE IT FAILS

Composability. Every operator presents the same interface, so any operator can be the child of any other with no coordination or special cases. New operators slot in without touching existing ones.

Pipelining. Tuples flow from leaves to root without materialising intermediate results, so memory consumption is bounded by the plan's depth rather than by the data size.

Natural laziness. A LIMIT at the top of the plan simply stops calling next(), and no operator below does any further work. Nothing special is required to make this happen.

And it makes one virtual call per tuple per operator. For an OLTP query touching ten rows through four operators, that is forty calls — irrelevant. **For an analytical query over a billion rows it is four billion indirect calls**, each defeating branch prediction, and that becomes the dominant cost of the query.

2 Beyond one tuple at a time

Model	Unit of work	Suits	Cost
Iterator (Volcano)	One tuple.	OLTP, and any query where latency to the first row matters.	One virtual call per tuple per operator.
Materialisation	The entire output of each operator, computed at once.	Small result sets; stored procedures; some embedded systems.	Every intermediate result must fit in memory.
Vectorised	A batch of roughly 1,000 tuples.	OLAP. The modern default for analytics.	More complex operators; slightly worse latency to the first row.

WHY BATCHES OF ABOUT A THOUSAND

Dispatch is amortised. One virtual call per thousand tuples rather than per tuple reduces the overhead by three orders of magnitude, to the point where it no longer appears in a profile.

The inner loop becomes tight and predictable. A loop over a thousand values of known type, with no indirect calls, is something the compiler can **auto-vectorise into SIMD instructions** — processing four, eight, or sixteen values per instruction.

A batch of a thousand values fits comfortably in L1 or L2 cache, so the data stays hot through the whole operation. Larger batches would spill to a slower level and the benefit would reverse — which is why the number is around a thousand rather than a million.

The combined effect is 10 to 100 times on analytical queries. This is the single most important execution idea of the last twenty years, and it is why column stores (Module 12) are fast: columnar storage supplies exactly the dense same-typed arrays that vectorised execution wants. **The two ideas belong together** and arriving at either tends to lead to the other.

COMPILATION

The alternative is to stop interpreting the plan and instead **generate machine code specialised to this exact query**, then execute that.

There is no dispatch at all, the tuple layout is known at code generation time so every field access is a constant offset, and the predicate is inlined rather than being an expression tree walked per row. The generated code resembles what a programmer would write by hand for this one query.

The cost is compilation time — from a few milliseconds to tens of milliseconds depending on the approach. **For a query that would have run in one millisecond, that is a large regression.**

So systems do both and choose adaptively: interpret short queries, compile long ones, and sometimes begin interpreting and switch to compiled code once the query proves to be long-running. HyPer compiles, DuckDB vectorises, and several systems combine the two.

3 Pipelines and breakers

A PIPELINE BREAKER MUST CONSUME ALL OF ITS INPUT FIRST

Pipelineable operators — filter, projection, index scan, the probe side of a hash join — take a tuple and may produce a tuple. They hold essentially no state and pass data through.

Pipeline breakers — sort, aggregation, DISTINCT, and the *build* side of a hash join — **cannot emit anything until they have seen their last input tuple**. A sort cannot know which row is first until it has seen all of them.

Breakers determine a plan's memory consumption, because they must accumulate their entire input. They are therefore where spilling happens (Module 06 §3), and where a cardinality misestimate turns into a cliff.

And they determine latency to the first row. A plan with a sort near the top produces no output at all until the sort has completed, regardless of how small the LIMIT is — which is why a LIMIT 10 query can take as long as the unlimited one.

- **Push breakers as late as possible.** A sort over the output of a selective filter is far cheaper than a sort over the base table, so the optimiser pushes filters below sorts whenever semantics allow.
- **A plan with no breakers streams:** constant memory, and rows emitted as soon as they are found. This is the ideal shape for a LIMIT query and is worth recognising in an EXPLAIN output.
- **The build side of a hash join is a breaker and the probe side is not.** This asymmetry is a concrete reason the optimiser cares which relation is which, beyond their sizes — the choice determines where the plan stalls and how much memory it needs.
- **Top-K avoids the sort breaker.** ORDER BY x LIMIT 10 can be executed with a bounded ten-element heap over a single streaming pass, which is not a breaker in the memory sense and changes the plan's profile entirely.
- **Counting the breakers in a plan** gives you its memory and latency profile at a glance, which is a useful habit when reading EXPLAIN output.

4 Parallelism

Kind	Mechanism	Assessment
Intra-operator	Partition the data and run the same operator on each partition concurrently.	The one that matters. This is what makes a single large query fast, and it scales with core count.
Inter-operator	Run different operators of the same plan on different cores, passing tuples between them.	Limited by pipeline depth — a plan has a handful of operators, so this caps out at a small speedup regardless of how many cores are available.
Inter-query	Run different queries on different cores.	Trivial, and every system does it. Does nothing for a single slow query.

MORSEL-DRIVEN PARALLELISM

Split the input into small chunks — morsels, typically tens of thousands of rows — and have worker threads pull them from a shared queue as they become free.

Work stealing handles skew automatically. A thread that finishes its morsel quickly simply takes another; a thread that gets an expensive morsel does not hold everyone up. No thread sits idle while another is still working.

It can be NUMA-aware: prefer handing a thread a morsel whose data is resident on its own socket's memory, falling back to remote morsels only when local ones are exhausted.

It replaced static partitioning, which divided the work evenly across threads in advance — and therefore divided it *badly* whenever the data was skewed, which is essentially always. The cost of a shared queue is far smaller than the cost of one thread doing three times its share.

- **Amdahl's law bounds everything.** The serial fraction of a query — planning, final aggregation, result assembly — limits the achievable speedup regardless of core count, and there is always a serial fraction.
- **Memory bandwidth is usually the real limit for scans**, not CPU. Adding cores that share the same memory controller does not add bandwidth, so a scan-bound query stops improving well before the cores are saturated. **This is a large part of why column stores and compression matter so much** (Module 12): they reduce the bytes that must cross the bottleneck.
- **Coordination is not free.** Exchanging tuples between threads involves queues, synchronisation, and cache-line transfers between cores.
- **Skew leaves cores idle** under static partitioning, which is precisely what morsels address.
- **A parallel plan has startup cost** — launching workers, setting up exchanges — so short queries run *slower* in parallel. This is why systems have a cost threshold below which they do not parallelise at all.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lectures 13–14, Query Execution	The iterator model, execution models, and parallelism. The primary source.
Graefe — Volcano, An Extensible and Parallel Query Evaluation System (free)	The iterator model of §1, in the original.
Boncz, Zukowski & Nes — MonetDB/X100: Hyper-Pipelining Query Execution (free)	The vectorised execution of §2, and the paper that established it.

Resource	What it covers for this module
Neumann — Efficiently Compiling Efficient Query Plans for Modern Hardware (free)	Query compilation, from HyPer. The counterpoint to vectorisation.
Leis et al. — Morsel-Driven Parallelism (free)	The scheduling approach of §4.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the iterator model with scan, filter, projection, and nested loop join operators. Run a query end to end.
- 2 Profile it on a hundred million rows and measure what fraction of time is virtual dispatch.
- 3 Implement vectorised versions of filter and projection operating on batches of 1024. Measure the speedup.
- 4 Vary the batch size from 1 to 100,000 and plot throughput. Identify where cache effects take over.
- 5 Confirm that the compiler auto-vectorises your batch filter by inspecting the generated assembly.
- 6 Identify the pipeline breakers in five query plans from EXPLAIN and predict each plan's memory behaviour.
- 7 Demonstrate the latency effect of a breaker: time to first row for a LIMIT 1 query with and without an ORDER BY.
- 8 Implement top-K for ORDER BY with LIMIT and compare against a full sort on a large table.
- 9 Implement intra-operator parallelism for a scan with static partitioning. Then introduce skew and show cores going idle.
- 10 Replace it with morsel-driven scheduling and show the skew handled.
- 11 Measure scan throughput against thread count and identify where memory bandwidth becomes the limit.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Describe the iterator interface and give three advantages of pull-based execution.
- 2 Why does the iterator model fail on analytical workloads but not OLTP?
- 3 Compare three execution models on unit of work, suitability, and cost.
- 4 Give three reasons vectorised execution is faster, and say why the batch size is about a thousand.

- 5 What does query compilation eliminate, what does it cost, and how do systems resolve that?
- 6 What is a pipeline breaker? Give four examples and two consequences.
- 7 Why is the build side of a hash join a breaker and the probe side not?
- 8 Name three kinds of parallelism and say which matters for one slow query.
- 9 What is morsel-driven parallelism and what does it fix?
- 10 Give four reasons parallel query execution disappoints.